Iterator Pattern

- Definition
- Real-world Usage
  - Code Example
- The Problem: Data exposure to the Client
- Class Diagram
- Structure of the Iterator Design Pattern
- Implementation(Example: Library)
  - Output

Resources

- 41. All Behavioral Design Patterns | Strategy, Observer, State, Template, Command, Visitor, Memento
- 33. Iterator Design Pattern Explained with Example | Low Level Design

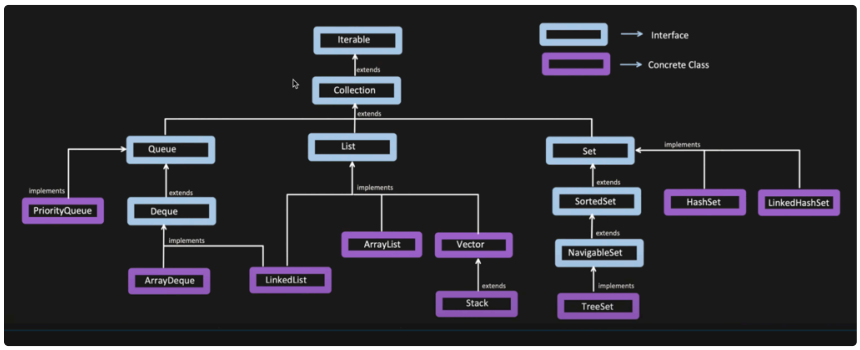
Definition

> **The Iterator design pattern is one of the behavioral design patterns that provides a way to access elements of a Collection sequentially without exposing the underlying representation of the collection.**

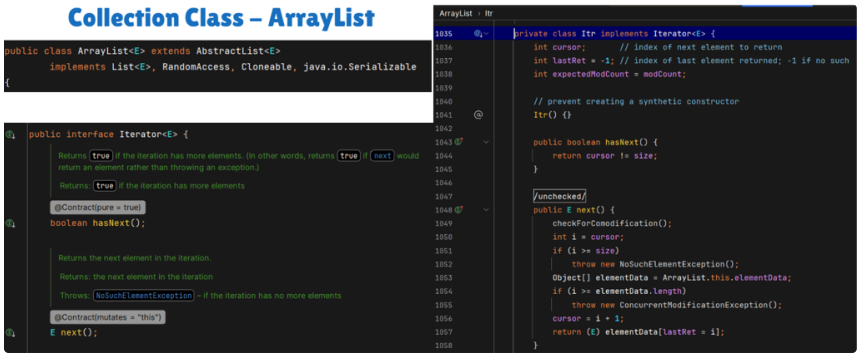
Concept && Coding

Real-world Usage

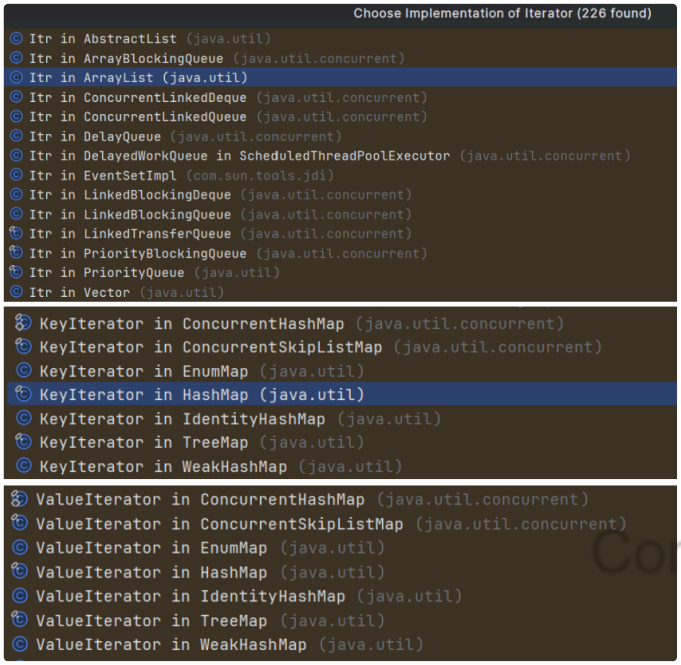
In the Java Collections Framework, the collection interface includes methods, such as `iterator()`, that allow a client to obtain an Iterator object from any collection that implements it. All collection classes have iterator implementation, e.g., (`ArrayList.iterator()` , `HashSet.iterator()` , `TreeSet.descendingIterator()`, etc).



Java Collections Framework



ArrayList implementation of Iterator



Concept && Coding

Iterator Interface Implementations

Code Example

```java
// Java Collections Usage Example
public class LinkedHashSetExample {
    public static void main(String[] args) {
        Set<Integer> intSet = new LinkedHashSet<>();
        intSet.add(2);
        intSet.add(77);
        intSet.add(82);
        intSet.add(63);
        intSet.add(5);

        // Common to all Collection Classes
        Iterator<Integer> iterable = intSet.iterator();
        while (iterable.hasNext()) {
            int value = iterable.next();
            System.out.println(value);
        }
    }
}
```

The Problem: Data exposure to the Client

```java
public class Book {
    private final String title;
    private final String author;
    private final String isbn;
```

```java

    public Book(String tittle, String author, String isbn) {
        this.title = tittle;
        this.author = author;
        this.isbn = isbn;
    }

    public static List<Book> getBooks() {
        List<Book> books = List.of(
                new Book("To Kill a Mockingbird", "Harper Lee", "978-0-74-7356-5"),
                new Book("The Great Gatsby", "F. Scott Fitzgerald", "778-0-24-7156-5"),
                new Book("The Catcher in the Rye", "J.D. Salinger", "333-0-28-7446-8"),
                new Book("The Hobbit", "J.R.R. Tolkien", "783-0-14-1951-8"),
                new Book("Rich Dad Poor Dad", "Robert Kiyosaki", "183-0-12-1491-8"),
                new Book("Pride and Prejudice", "Jane Austen", "289-0-12-1678-8")
        );
        return books;
    }

    @Override
    public String toString() {
        return "Book [" + "Tittle='" + title + ", Author=" + author + ", age=" + "ISBN=" + isbn + "]";
    }
}
```

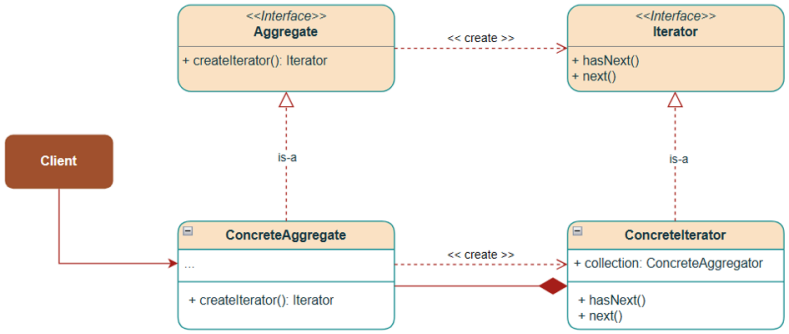
```java
public class Client {
    public static void main(String[] args) {
        System.out.println("\n###### Problem without Iterator Pattern Demo ######");
        // Client has access to the entire book list in the library
        List<Book> bookList = Book.getBooks();
        for (Book book : bookList) {
            System.out.println(book);
        }
    }
}
```

Concept && Coding

Problems with the above code are that:

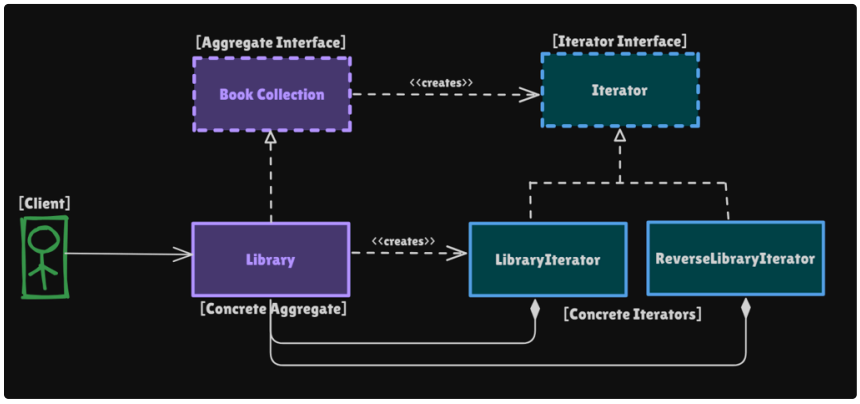
- No data encapsulation allows the client can modify data and access hidden information.
- Tight coupling of traversal logic to data structure and client.
- Violates SOLID principles if we try to implement new traversal logic.

Class Diagram



Structure of the Iterator Design Pattern

Let's understand the Structure of the Iterator Design Pattern using a Library example:



- **Iterator Interface(** `Iterator<T>` **)**: Defines the contract for traversing a collection with `hasNext()` and `next()` methods.
- **Concrete Iterator**: Implements the traversal logic for a specific collection type. It tracks the current position while traversing the collection.
  - `LibraryIterator` → traverses books from first to last.
  - `DescendingLibraryIterator` → traverses books from last to first by starting at `numberOfBooks - 1` and decrementing the position.
- **Aggregate Interface(** `BookCollection` **)**: Declares a method to create iterators (`createIterator()` and `createReverseIterator()`).
- **Concrete Aggregate(** `Library` **)**: The actual collection of books (`Library` class) that implements the aggregate interface(creates and returns **Concrete Iterators** `LibraryIterator` **and** `DescendingLibraryIterator`).

Concept && Coding

Implementation(Example: Library)

```java
// Book class representing individual books in a library
public class Book {
    private final String title;
    private final String author;
    private final String isbn;
    private int price;

    public Book(String title, String author, String isbn) {
        this.title = title;
        this.author = author;
        this.isbn = isbn;
    }

    public String getTitle() {
        return title;
    }

    public String getAuthor() {
        return author;
    }

    public String getIsbn() {
        return isbn;
    }
```

```java

    public int getPrice() {
        return price;
    }

    @Override
    public String toString() {
        return "Book [Title=" + title + ", Author=" + author + ", ISBN=" + isbn + "]";
    }
}
```

```java
// Aggregate interface
public interface BookCollection {
    Iterator<Book> createIterator();

    Iterator<Book> createReverseIterator();
}
```

```java
// Iterator interface
public interface Iterator<T> {
    boolean hasNext();

    T next();
}
```

```java
// Concrete Aggregate
public class Library implements BookCollection {

    private final List<Book> books;

    public Library(List<Book> books) {
        this.books = books;
    }

    @Override
    public Iterator<Book> createIterator() {
        return new LibraryIterator(books);
    }

    @Override
    public Iterator<Book> createReverseIterator() {
        return new ReverseLibraryIterator(books);
    }
}
```

Concept && Coding

```java
// Concrete Iterator - for Library
public class LibraryIterator implements Iterator<Book> {
    private final List<Book> books;
    private int position = 0;

    public LibraryIterator(List<Book> books) {
        this.books = books;
    }

    @Override
    public boolean hasNext() {
        return position < books.size();
    }

    @Override
    public Book next() {
        return books.get(position++);
    }
}
```

```java
// Concrete Iterator - for Library
public class ReverseLibraryIterator implements Iterator<Book> {
    private final List<Book> books;
    private int position;
```

```java

    public ReverseLibraryIterator(List<Book> books) {
        this.books = books;
        this.position = books.size() - 1;
    }

    @Override
    public boolean hasNext() {
        return position >= 0 && books.get(position) != null;
    }

    @Override
    public Book next() {
        if (!hasNext()) {
            return null;
        }
        return books.get(position--); // Return current book and move backward
    }
}
```

```java
// Client
public class LibraryIteratorDemo {

    public static void main(String[] args) {
        System.out.println("\n###### Iterator Design Pattern ######");

        // Create Library
        Library library = getLibrary();

        // Forward iteration
        Iterator<Book> iterator = library.createIterator();
        System.out.println("\n==> Forward iteration:");
        displayLibrary(iterator);

        // Reverse iteration
        Iterator<Book> reverseIterator = library.createReverseIterator();
        System.out.println("\n==> Reverse iteration:");
        displayLibrary(reverseIterator);
    }

    private static Library getLibrary() {
        List<Book> books = List.of(
                new Book("To Kill a Mockingbird", "Harper Lee", "978-0-74-7356-5"),
                new Book("The Great Gatsby", "F. Scott Fitzgerald", "778-0-24-7156-5"),
                new Book("The Catcher in the Rye", "J.D. Salinger", "333-0-28-7446-8"),
                new Book("The Hobbit", "J.R.R. Tolkien", "783-0-14-1951-8"),
                new Book("Rich Dad Poor Dad", "Robert Kiyosaki", "183-0-12-1491-8"),
                new Book("Pride and Prejudice", "Jane Austen", "289-0-12-1678-8")
        );
        Library library = new Library(books);
        return library;
    }

    private static void displayLibrary(Iterator<Book> iterator) {
        while (iterator.hasNext()) {
            Book book = iterator.next();
            System.out.println(book);
        }
    }
}
```

Concept && Coding

Output

```
###### Iterator Design Pattern ######

==> Forward iteration:
Book [Title=To Kill a Mockingbird, Author=Harper Lee, ISBN=978-0-74-7356-5]
Book [Title=The Great Gatsby, Author=F. Scott Fitzgerald, ISBN=778-0-24-7156-5]
Book [Title=The Catcher in the Rye, Author=J.D. Salinger, ISBN=333-0-28-7446-8]
Book [Title=The Hobbit, Author=J.R.R. Tolkien, ISBN=783-0-14-1951-8]
Book [Title=Rich Dad Poor Dad, Author=Robert Kiyosaki, ISBN=183-0-12-1491-8]
Book [Title=Pride and Prejudice, Author=Jane Austen, ISBN=289-0-12-1678-8]

==> Reverse iteration:
Book [Title=Pride and Prejudice, Author=Jane Austen, ISBN=289-0-12-1678-8]
Book [Title=Rich Dad Poor Dad, Author=Robert Kiyosaki, ISBN=183-0-12-1491-8]
Book [Title=The Hobbit, Author=J.R.R. Tolkien, ISBN=783-0-14-1951-8]
Book [Title=The Catcher in the Rye, Author=J.D. Salinger, ISBN=333-0-28-7446-8]
Book [Title=The Great Gatsby, Author=F. Scott Fitzgerald, ISBN=778-0-24-7156-5]
Book [Title=To Kill a Mockingbird, Author=Harper Lee, ISBN=978-0-74-7356-5]

Process finished with exit code 0
```

Concept && Coding